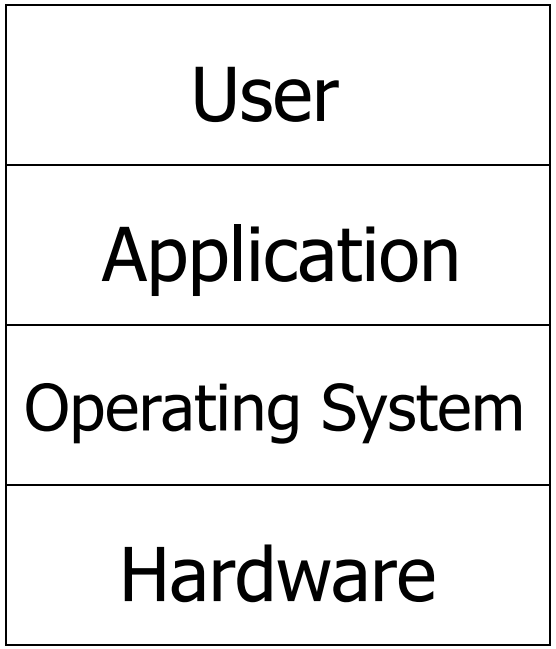
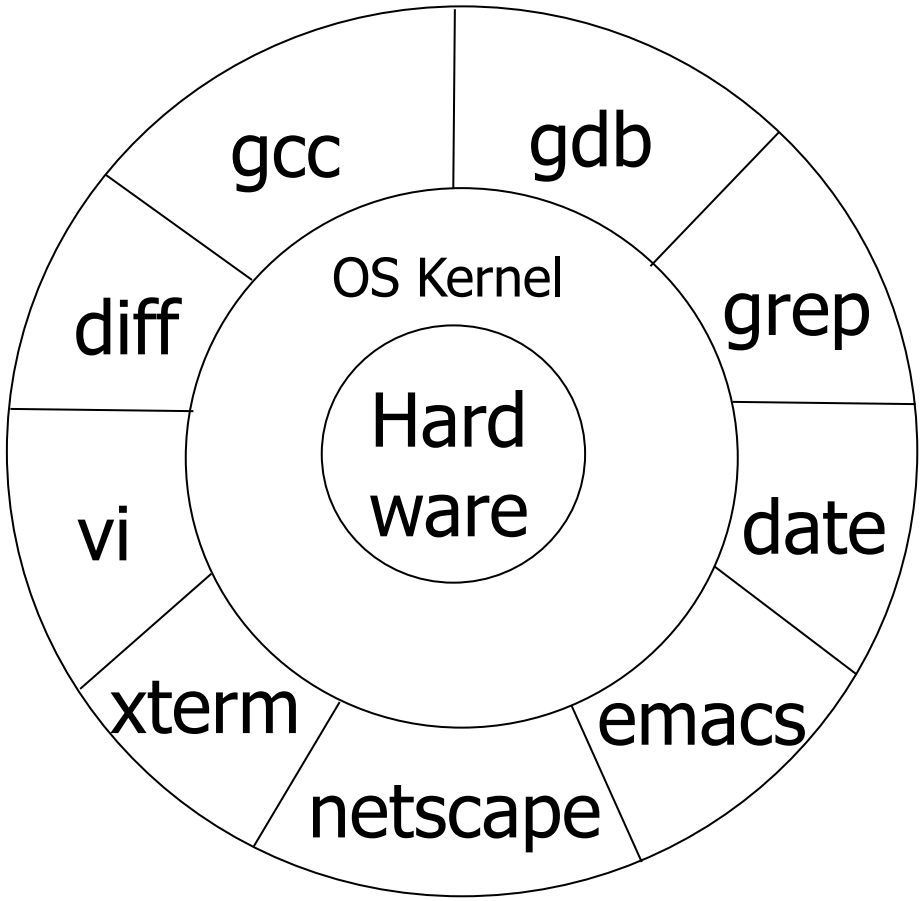
National University of Computer & Emerging Sciences

Lecture 3

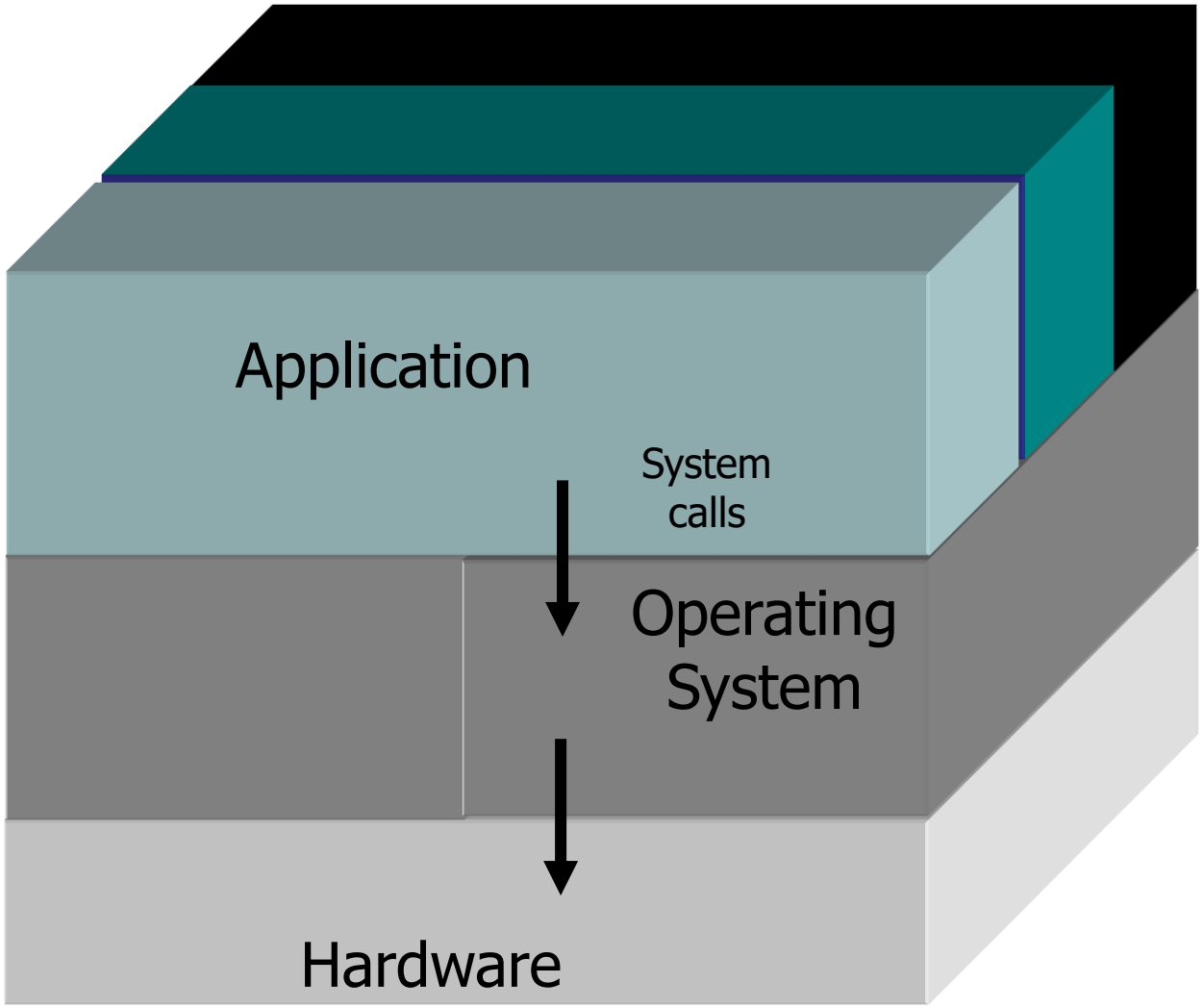
OS Overview

The Operating System controls the machine





A better picture



Many applications

One Operating System

One Hardware

Operating System in Action

- OS is a program, just like any other program
- When you turn power on, ***bootstrap*** program is loaded from ROM
- Boot program
  - Examine/check machine configuration
    - # CPUs
    - How much memory
    - # and type of HW devices
  - Build configuration structure describing the HW
  - Locates and Loads the OS
  - The control transfers to the OS

Operating System in Action

Operating System:

- Initialize kernel data structures
- Initialize state of HW devices
- Creates a number of ***processes*** to start operation

Components of an OS

Common System Components

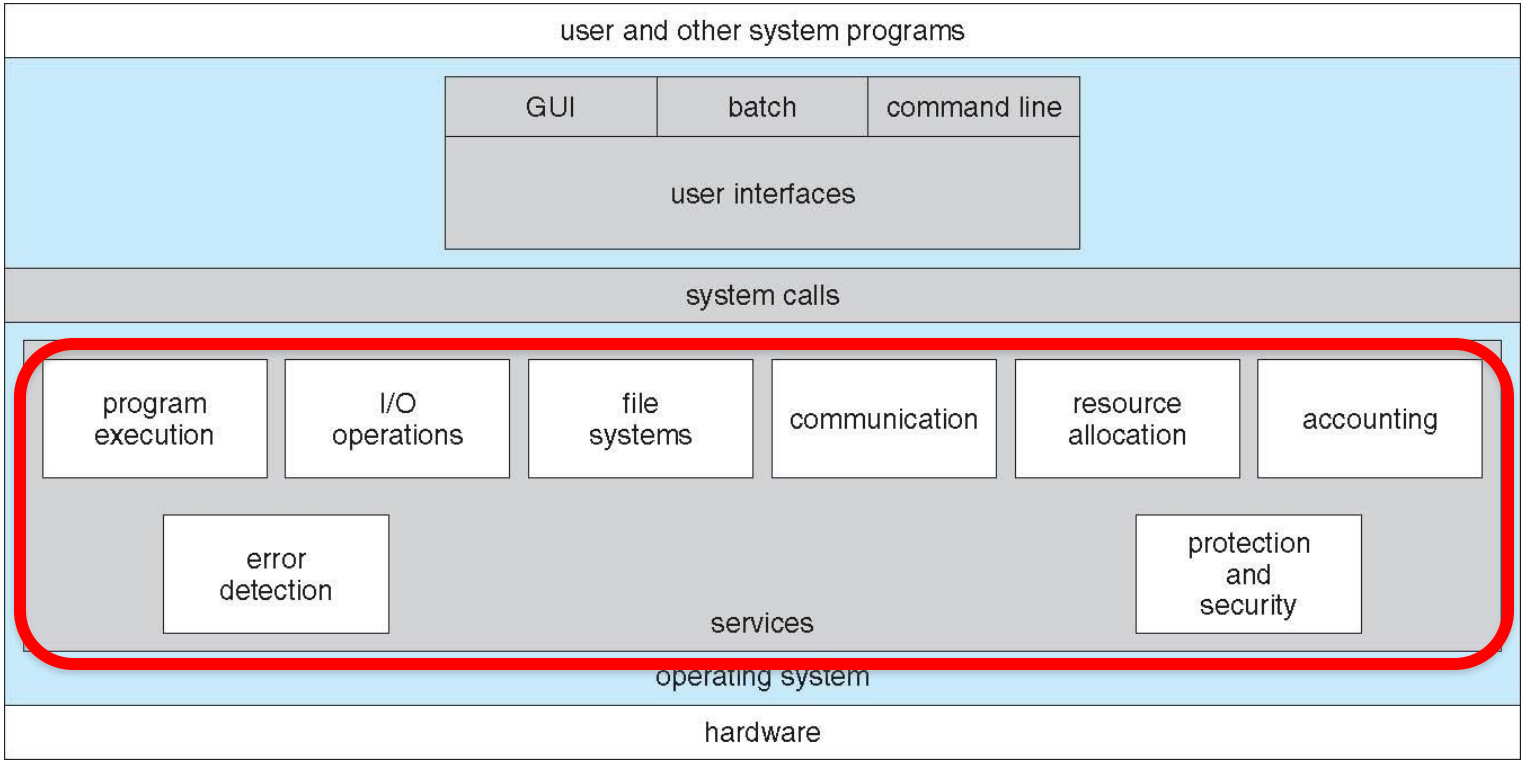
- Process Management
- Main Memory Management
- File Management
- I/O System Management
- Secondary Management
- Networking
- Protection System
- Command-Interpreter System

Process Management

- Creation/deletion of a process
- Create and make available, an execution environment
- Suspension/resumption and CPU scheduling in a fair manner
- Mechanisms for synchronization
- Mechanisms for inter-process communication
- Deadlock handling

Operating System Services

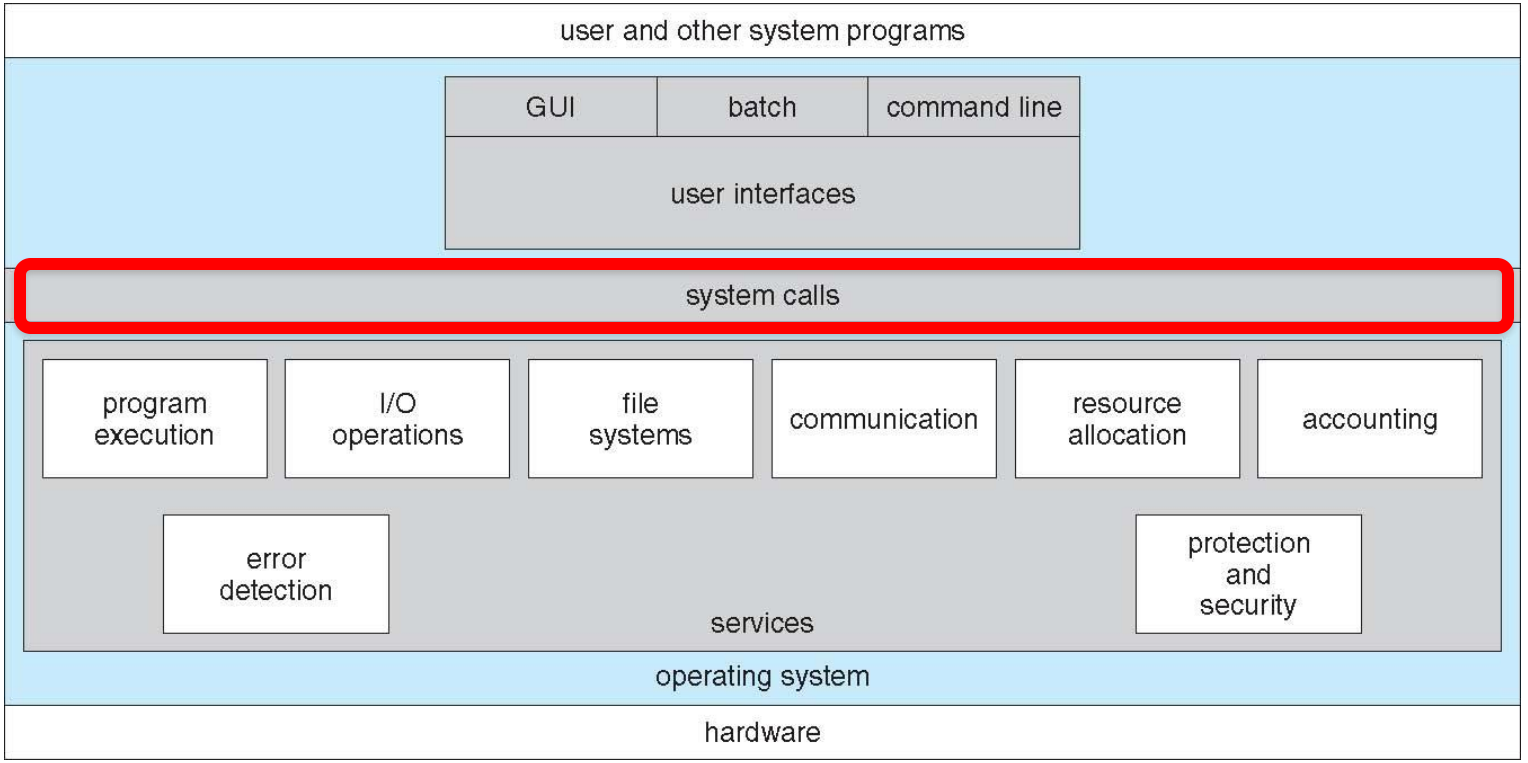
A View of Operating System Services



Operating System Services

- **Program execution** – system capability to load a program into memory and to run it
- **I/O operations** – since user programs cannot execute I/O operations directly, the operating system must provide some means to perform I/O
- **File-system manipulation** – program capability to read, write, create, and delete files
- **Communications** – exchange of information between processes executing either on the same computer or on different systems tied together by a network. Implemented via *shared memory* or *message passing*
- **Error detection** – ensure correct computing by detecting errors in the CPU and memory hardware, in I/O devices, or in user programs

A View of Operating System Services

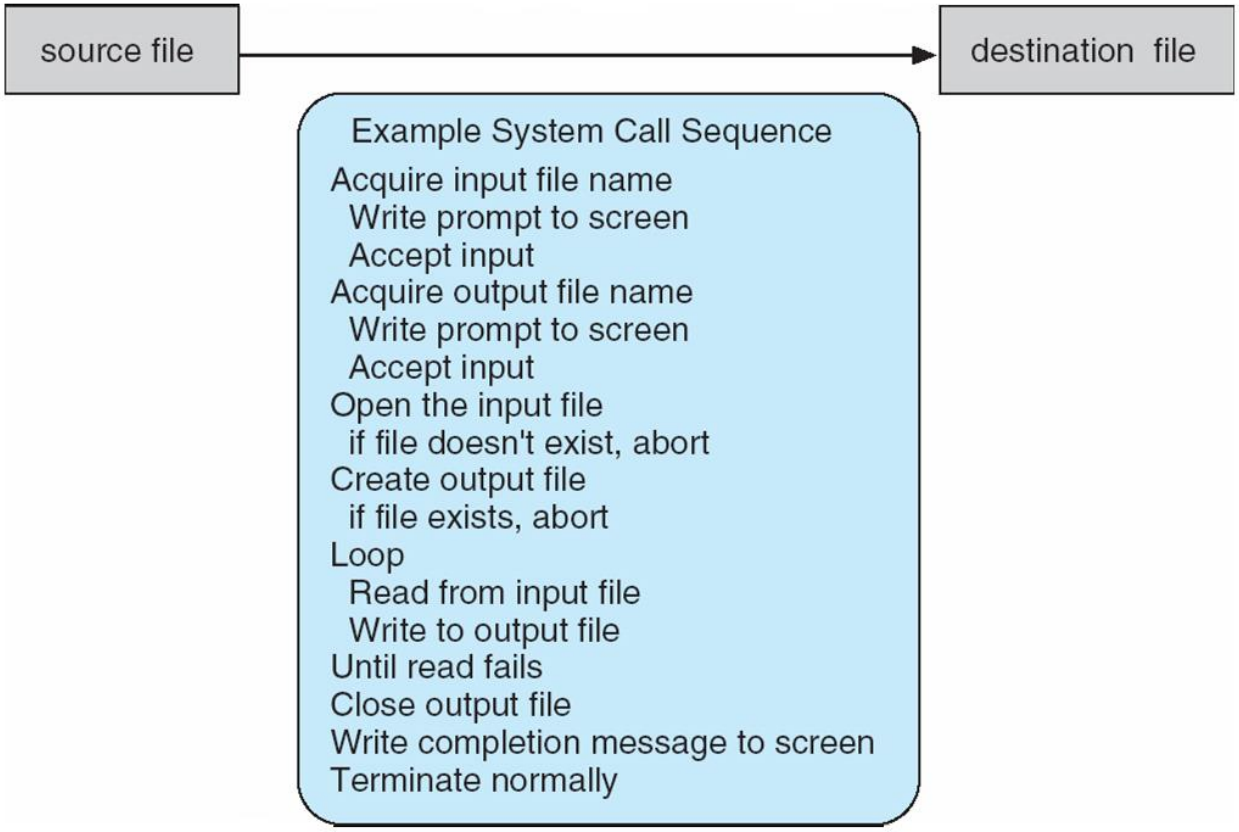


System Calls

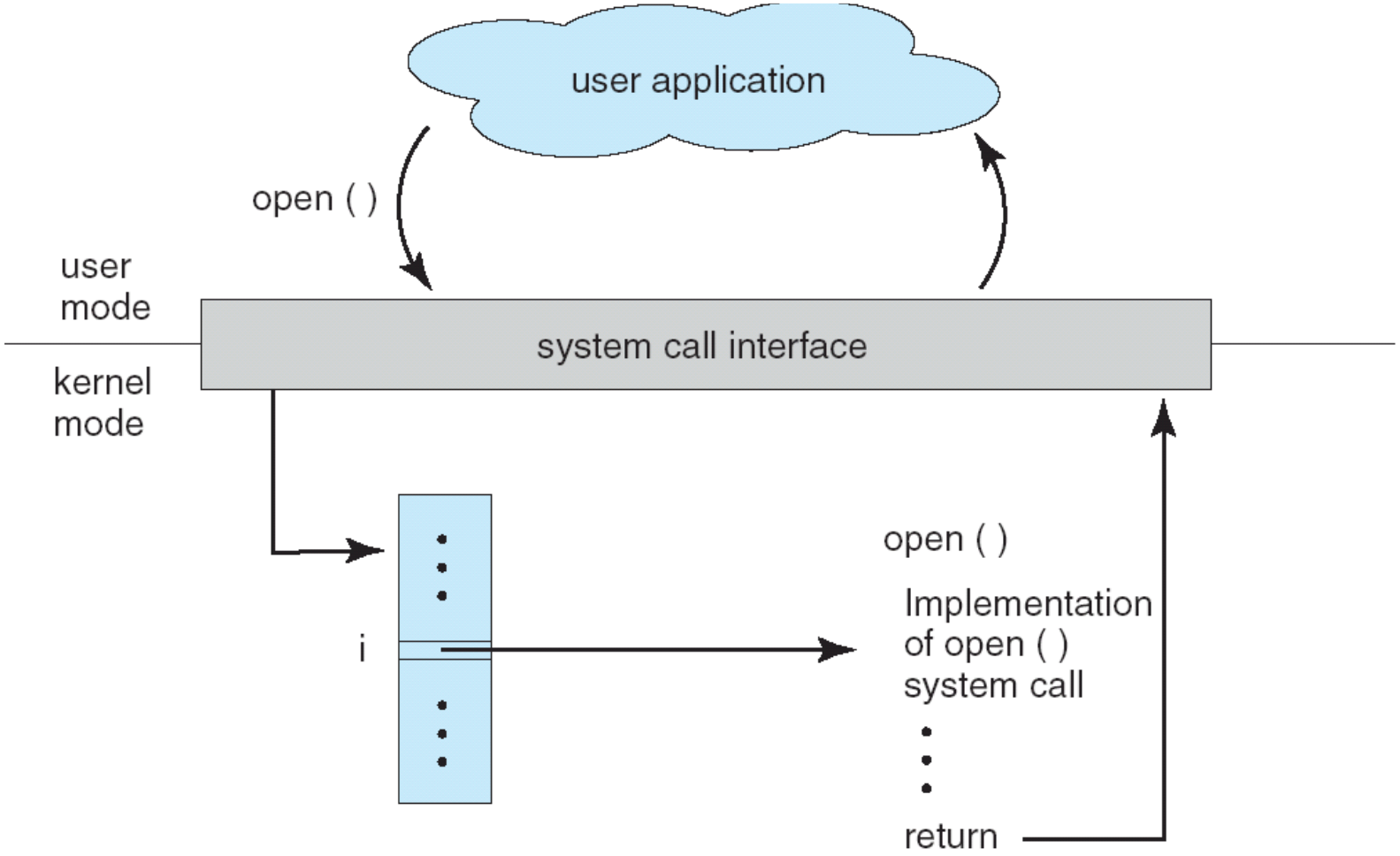
- Programming interface to the services provided by the OS
- Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level Application Program Interface (API) rather than direct system call use
- Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM)

Example of System Calls

- System call sequence to copy the contents of one file to another file



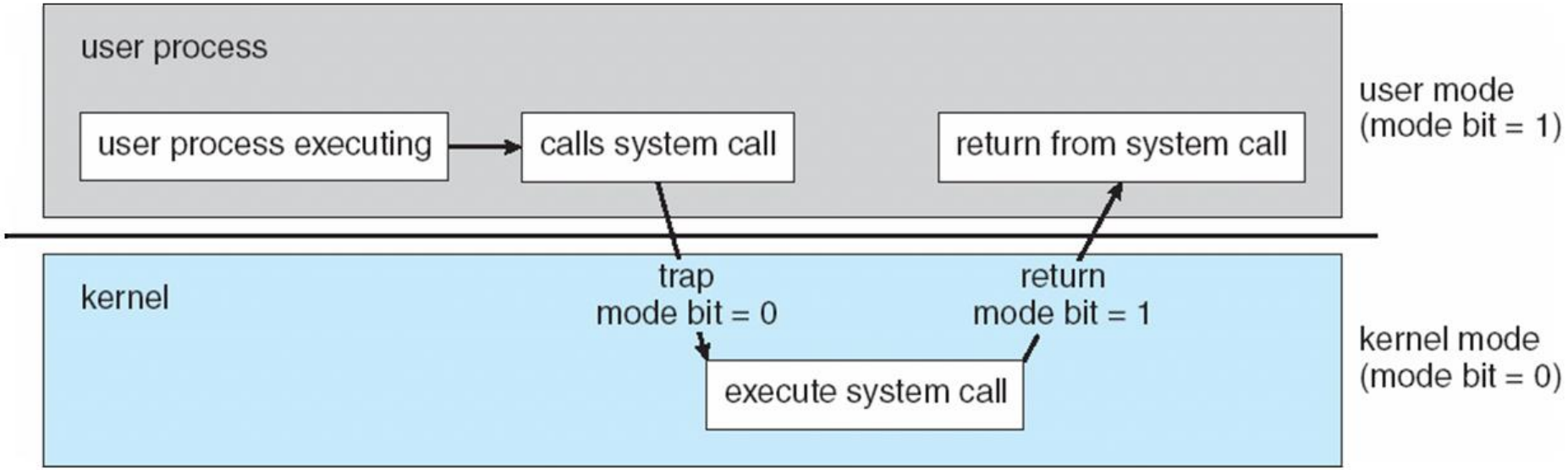
API – System Call – OS Relationship



Dual-mode operation

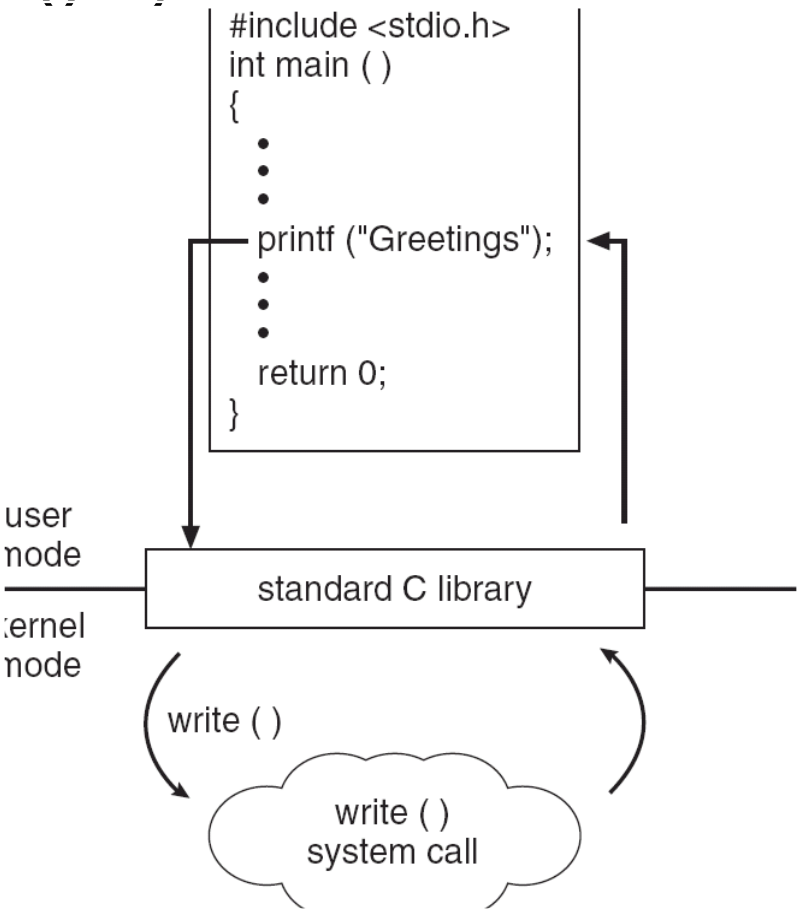
- **Dual-mode** operation allows OS to protect itself and other system components
  - **User mode** and **kernel mode**
  - **Mode bit** provided by hardware
    - Provides ability to distinguish when system is running user code or kernel code
    - Some instructions designated as **privileged**, only executable in kernel mode
    - System call changes mode to kernel, return from call resets it to user

Transition from User to Kernel Mode



Standard C Library Example

- C program invoking printf() library call, which calls write() system call



System Call Parameter Passing

- methods used to pass parameters to the OS
  - Simplest: pass the parameters in *registers*
    - In some cases, may be more parameters than registers
  - Parameters stored in a *block,* or table, in memory, and address of block passed as a parameter in a register
    - This approach taken by Linux and Solaris
  - Parameters placed, or *pushed,* onto the *stack* by the program and *popped* off the stack by the operating system
  - Block and stack methods do not limit the number or length of parameters being passed

Types of System Calls

- Process control
- File management
- Device management
- Information maintenance
- Communications
- Protection

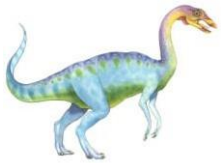
Examples of Windows and Unix System Calls

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitlializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

Operating System Design and Implementation

Simple Structure

- MS-DOS – written to provide the most functionality in the least space
  - Not divided into modules
  - Although MS-DOS has some structure, its interfaces and levels of functionality are not well separated

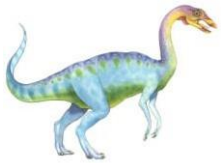


Non Simple Structure -- UNIX

UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring. The UNIX OS consists of two separable parts

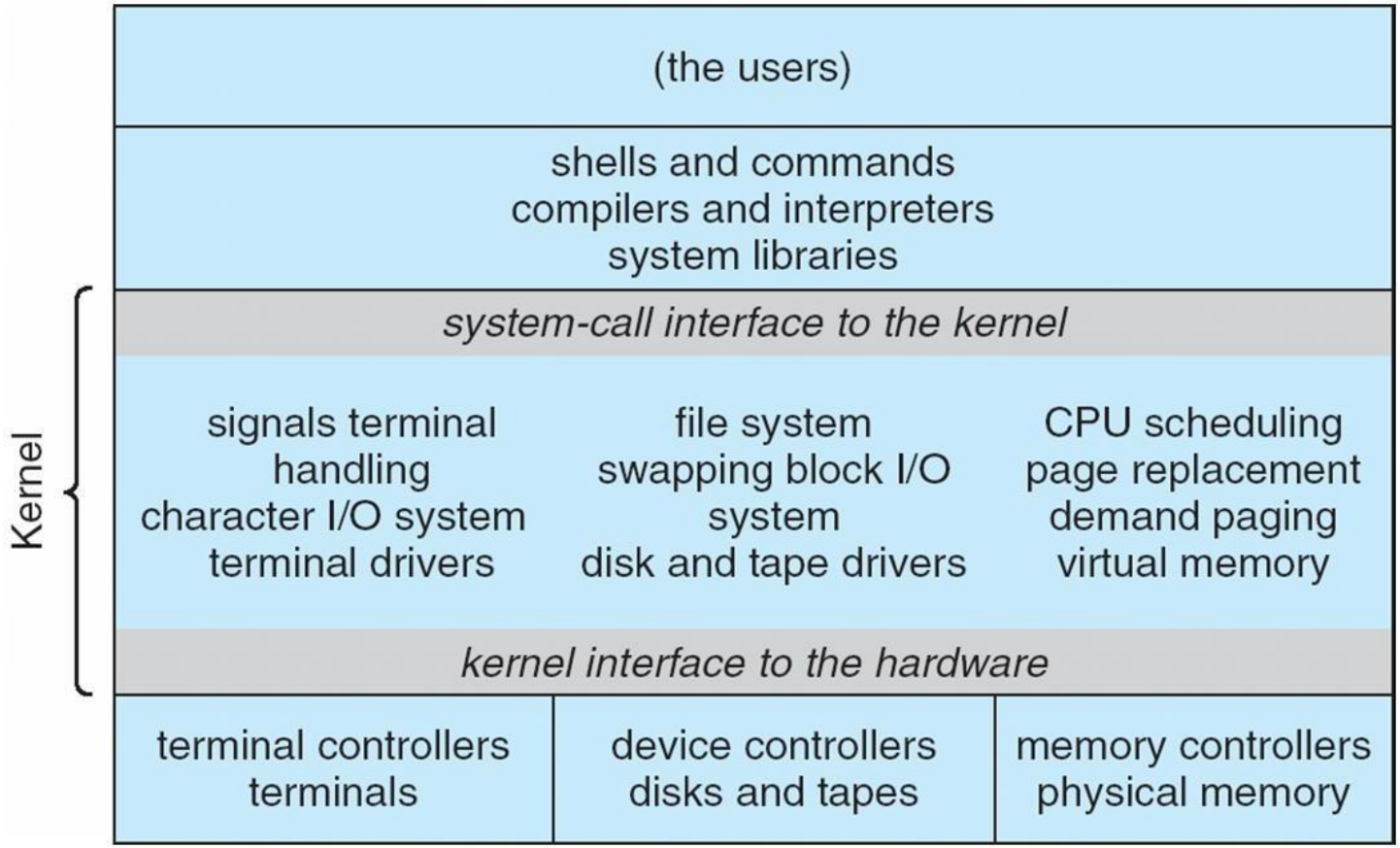
- Systems programs
- The kernel
  - Consists of everything below the system-call interface and above the physical hardware
  - Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level





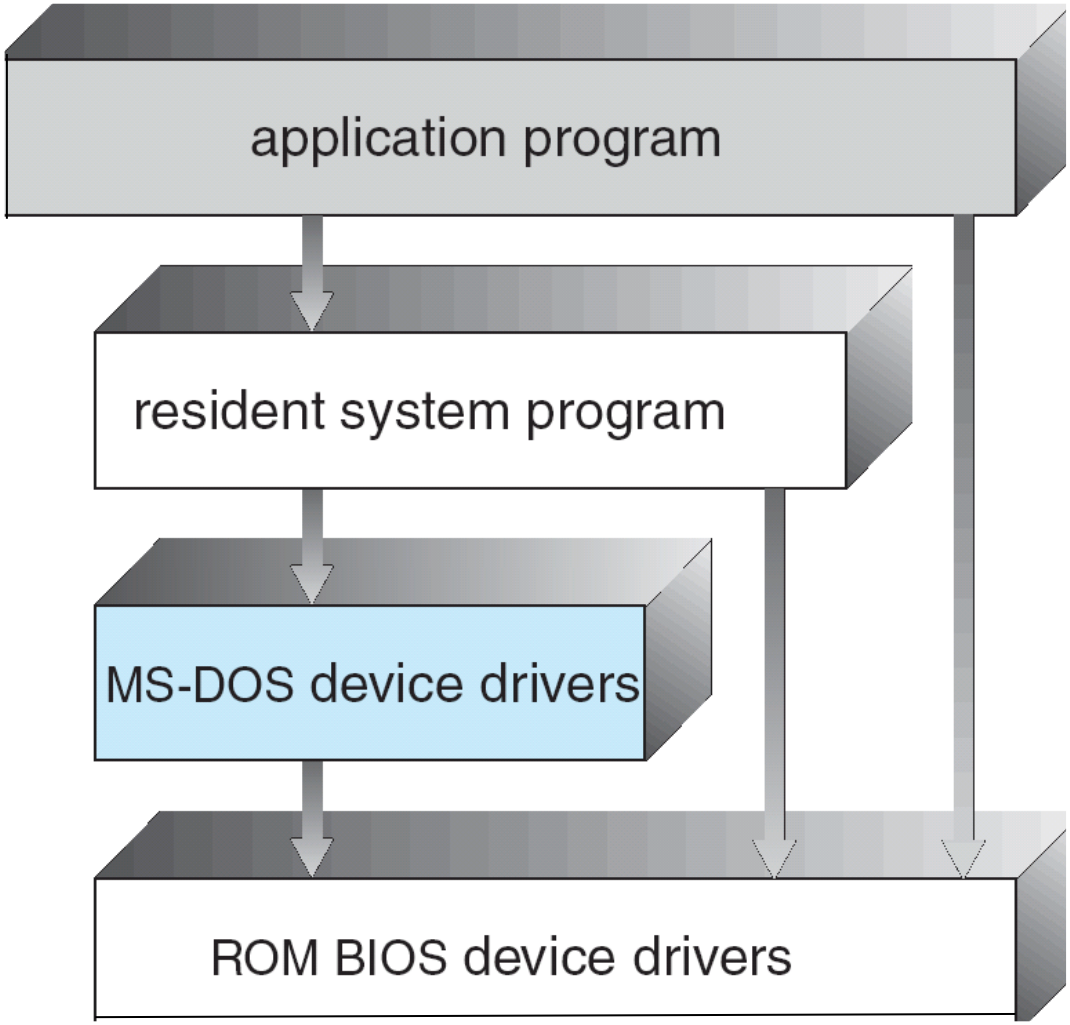
Traditional/Monolithic UNIX System Structure

Beyond simple but not fully layered

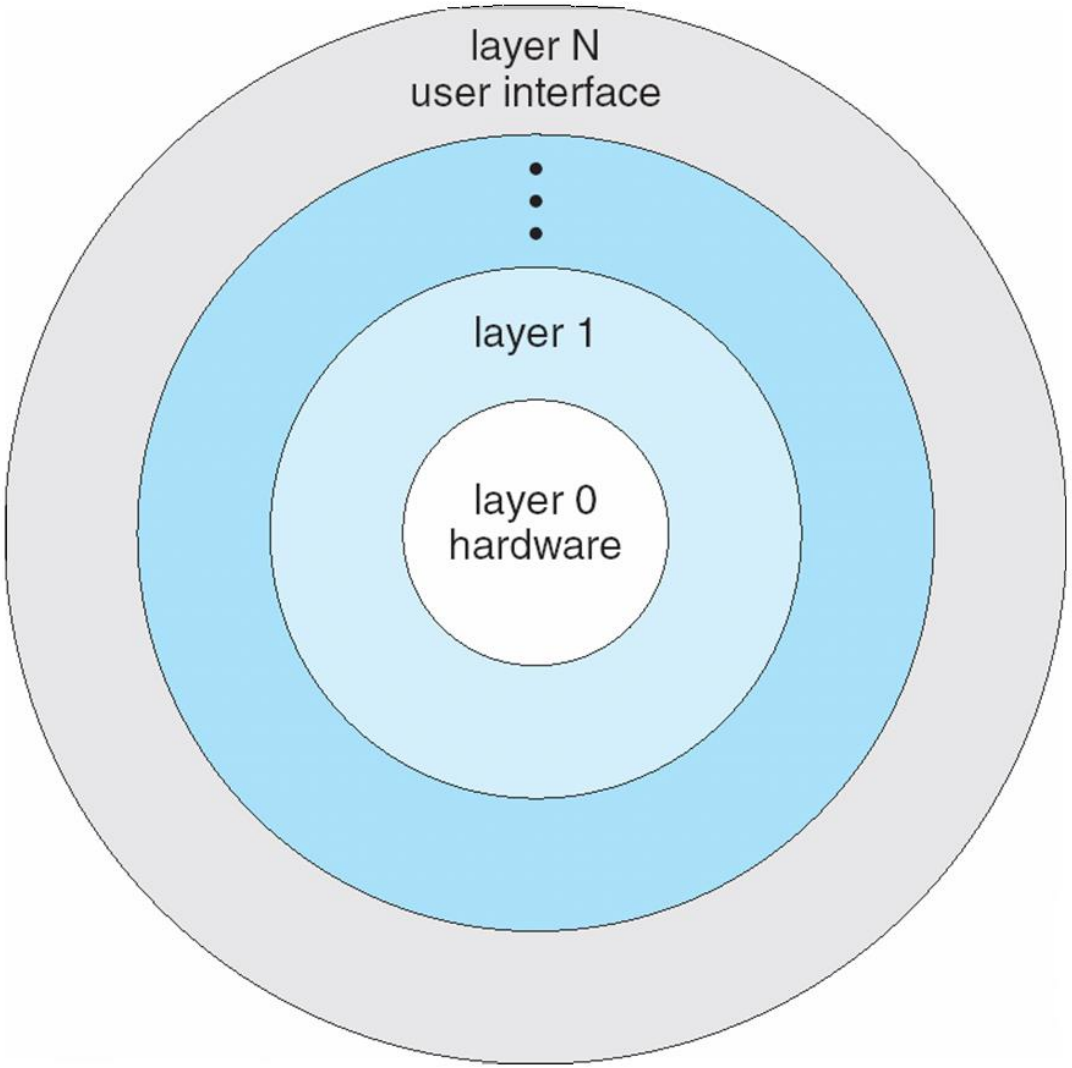




MS-DOS Layer Structure

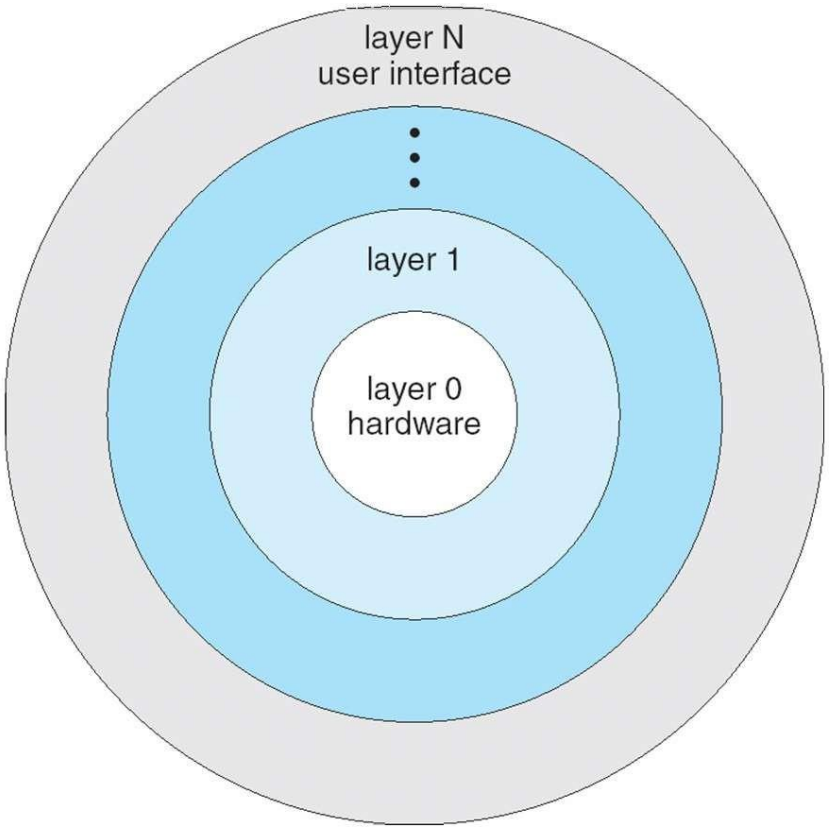


Layered Operating System

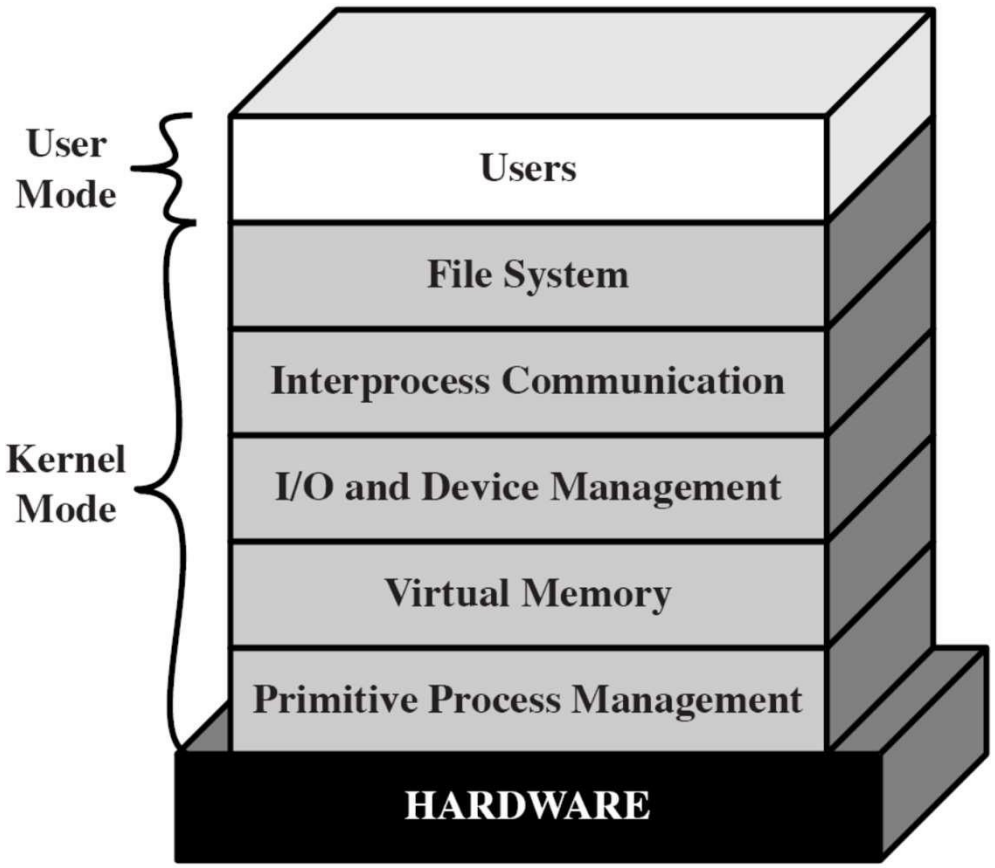


Layered Approach

- Break operating system into pieces
- Each layer interacts only with its lower layer
- Benefits
  - Allows to built modular systems
  - Can exchange each layer
  - Facilitates testing
    - Look at each layer separately
- Drawback
  - Difficult to define appropriate layers
  - Sometimes contradictory needs
  - Tends to be less efficient



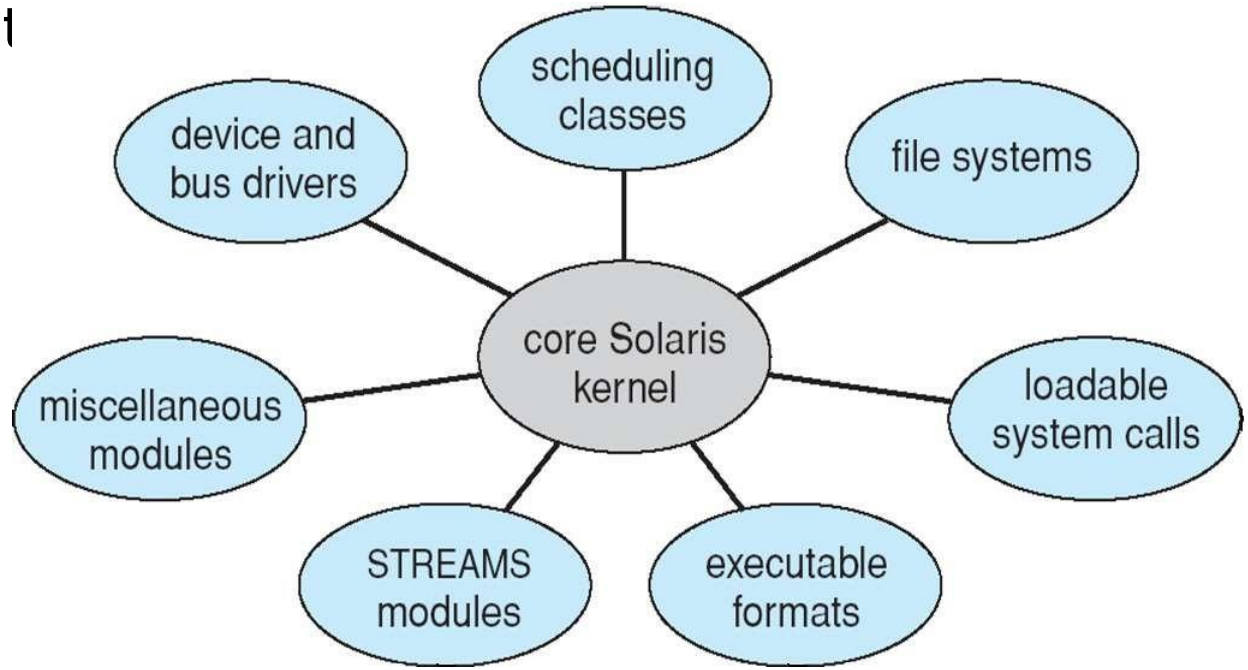
Layered Approach – Example



Modular Approach

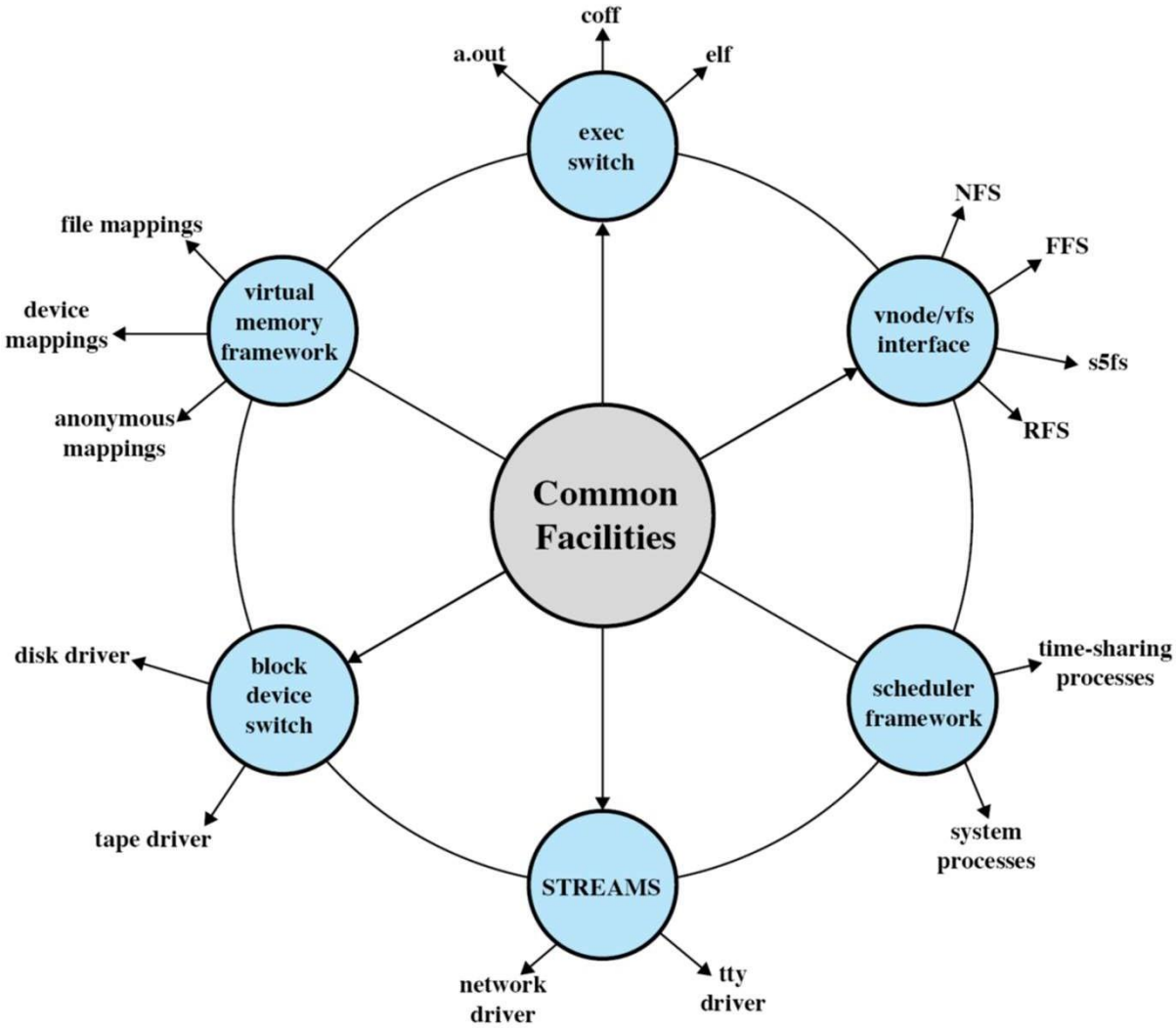
Solaris Modular Approach

- Most modern operating systems implement kernel modules
  - Each core component is separate
  - Each talks to the others over known interfaces
  - Each is loadable as needed within the kernel
- Overall, similar t

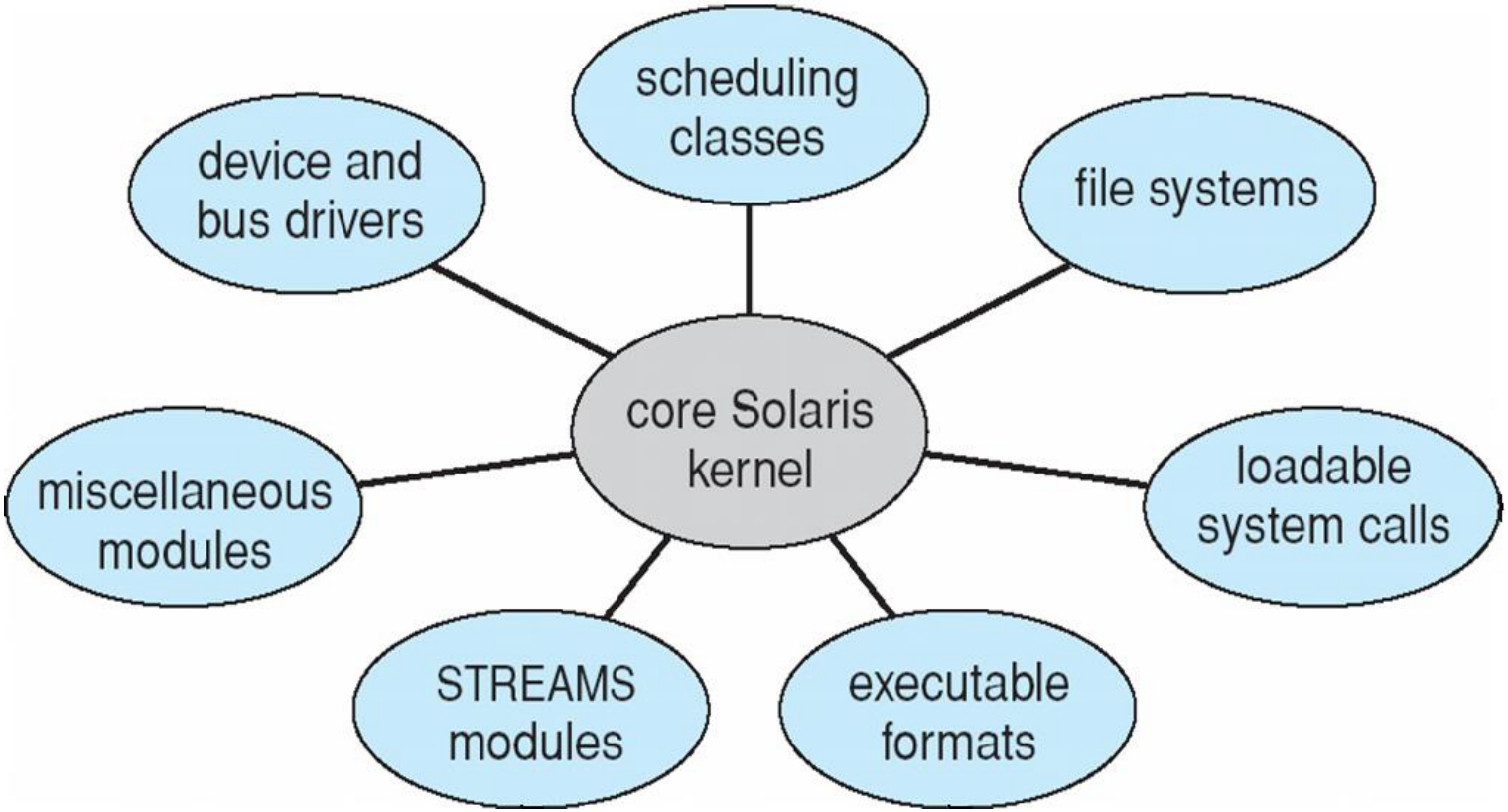


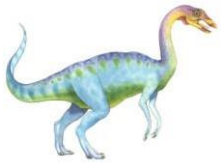
Modular Approach in Modern UNIX Instances

- Kernel set of core facilities
- Modules can be dynamically linked during run time
- Modern Unix approaches, e.g., Solaris follow a modular organization



Solaris Modular Approach





Microkernel System Structure

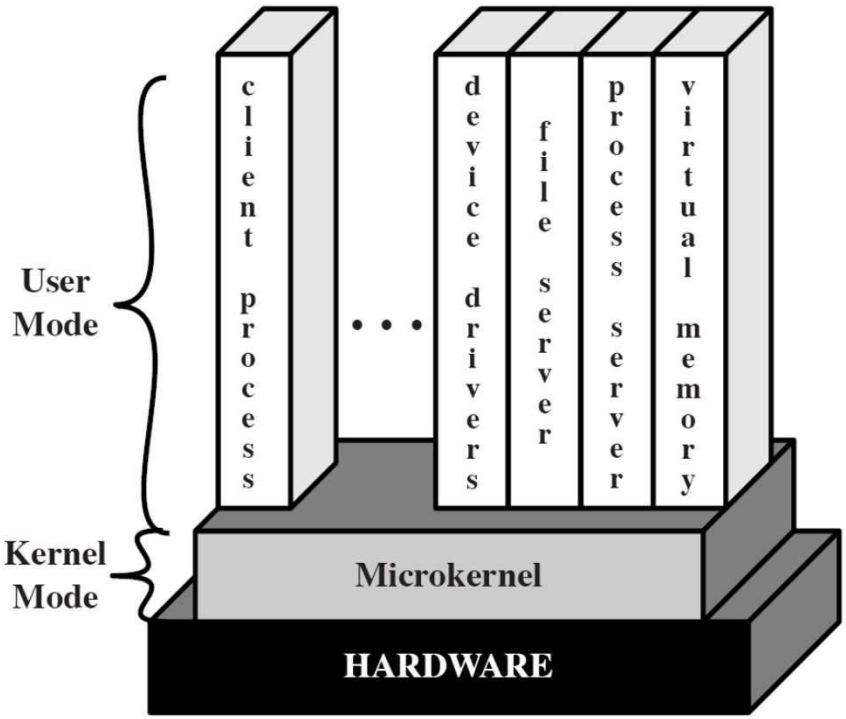
- Microkernels take “separation of policy and mechanism” approach to its furthest extent
  - Generally, any component that implements a system policy should be implemented as a user-mode process
- Moves as much from the kernel into user space
- **Mach** example of **microkernel**
  - Mac OS X kernel (**Darwin**) partly based on Mach
- Communication takes place between user modules using **message passing**
- Benefits:
  - Easier to extend a microkernel
  - Easier to port the operating system to new architectures
  - More reliable (less code is running in kernel mode) and More secure
- Detriments:
  - Performance overhead of user space to kernel space communication

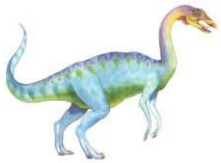


Microkernel

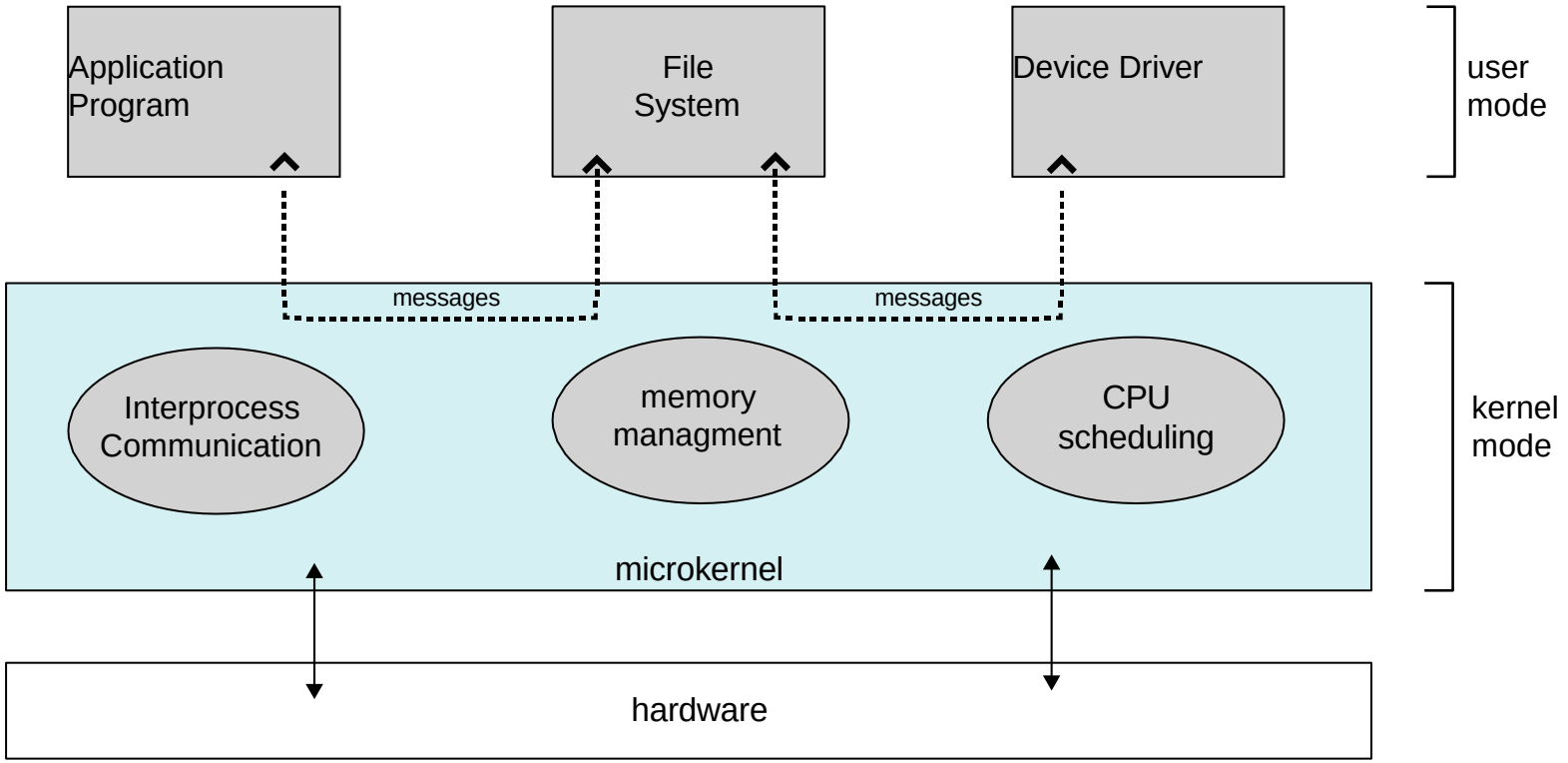
- Small OS core – contains only essential OS functions
  - Low-level memory management
  - Process scheduling
  - Communication facility

- Many services traditionally included in the OS kernel are now external subsystems
  - Device drivers, file systems, virtual memory manager, windowing system, security services
  - Run in user mode





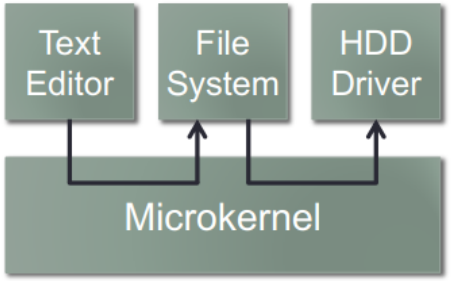
Microkernel System Structure





Microkernel Example

- Example: use a text editor to create and save a file
- Text editor is a user-mode process (duh)
- Filesystem implementation and HDD driver are also user-mode processes
- When text editor wants to create a file, it sends an IPC message to the filesystem service



- Filesystem service interacts with HDD driver via IPC to create the file on the physical disk
- When file has been created (or in event of failure), filesystem service sends an IPC message back to editor

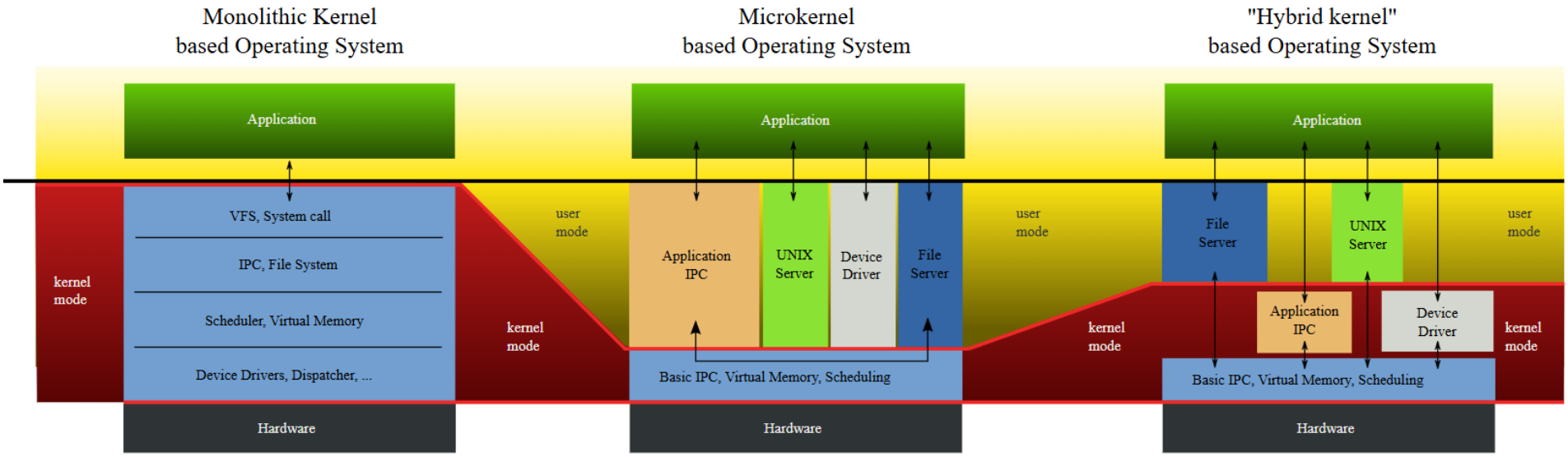
Hybrid Kernels

- Microkernels have performance issues.
- Important OS services live in user space:
- Interactions between multiple services require context-switches
- IPC through the kernel requires switching between kernel mode and user mode
- **Hybrid kernels** retain the same conceptual structure of microkernels, but move some services into the kernel
  - High degree of modularity, even for services within the kernel
  - Some services continue to be provided by user-mode processes, allowing for easy extension of the operating system
  - Of course, they also lose some reliability benefits of microkernels
- macOS (prev. Mac OS X) XNU kernel is a hybrid kernel
  - Based on Mach 3.0, with BSD facilities migrated into the kernel

Hybrid Kernels

- A hybrid Kernel combines elements of both monolithic and microkernel architectures.
- It also has modules or processes that run in user mode
- communicate with the kernel and each other through various mechanisms, such as message passing, shared memory, or direct function calls.
- balance the trade-offs between performance and modularity, as it can optimize the communication or switching between different components depending on the situation.
- The disadvantage is that it is more complex and inconsistent, as it may have different design principles or interfaces for different components.
- Examples of hybrid operating systems are
    - BeOS
    - Haiku
    - ReactOS
    - ChourosOS

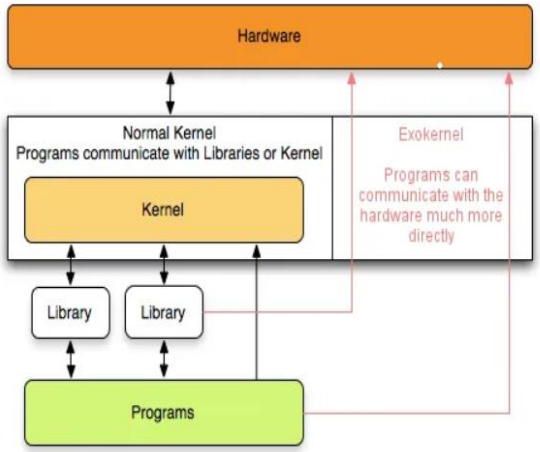
Comparison



One prominent example of a hybrid kernel is Windows 11 and Windows Server 2022, and powers Windows Phone 8, Windows Phone 8.1, Windows 10 Mobile, and the Xbox One and Xbox Series consoles.

ExoKernels

- Exokernels extend microkernel principles for extreme customization.
- intentionally expose lower-level hardware access by default
- Applications are given direct access to hardware resources, allowing them to manage resources and make decisions that the kernel previously made.
- One or more “Library OSes” or libOSes run on top of the exokernel
- • Provides additional abstractions on top of the computer hardware Applications run on top of a specific libOS
- direct communication channels to disk, network, memory, and processors.



Examples
Nemesis OS
MIT Exokernel Operating System

NanoKernels

A nanokernel is an ultra-lightweight and Extremely small kernel that provides only essential low-level hardware management and IPC.

Key Characteristics:

Minimalist design – only basic hardware abstraction

Low overhead – efficient and resource-friendly

High modularity – most services run in user space

Real-time capabilities – predictable execution

UseCases:

Embedded & IoT devices (low power)

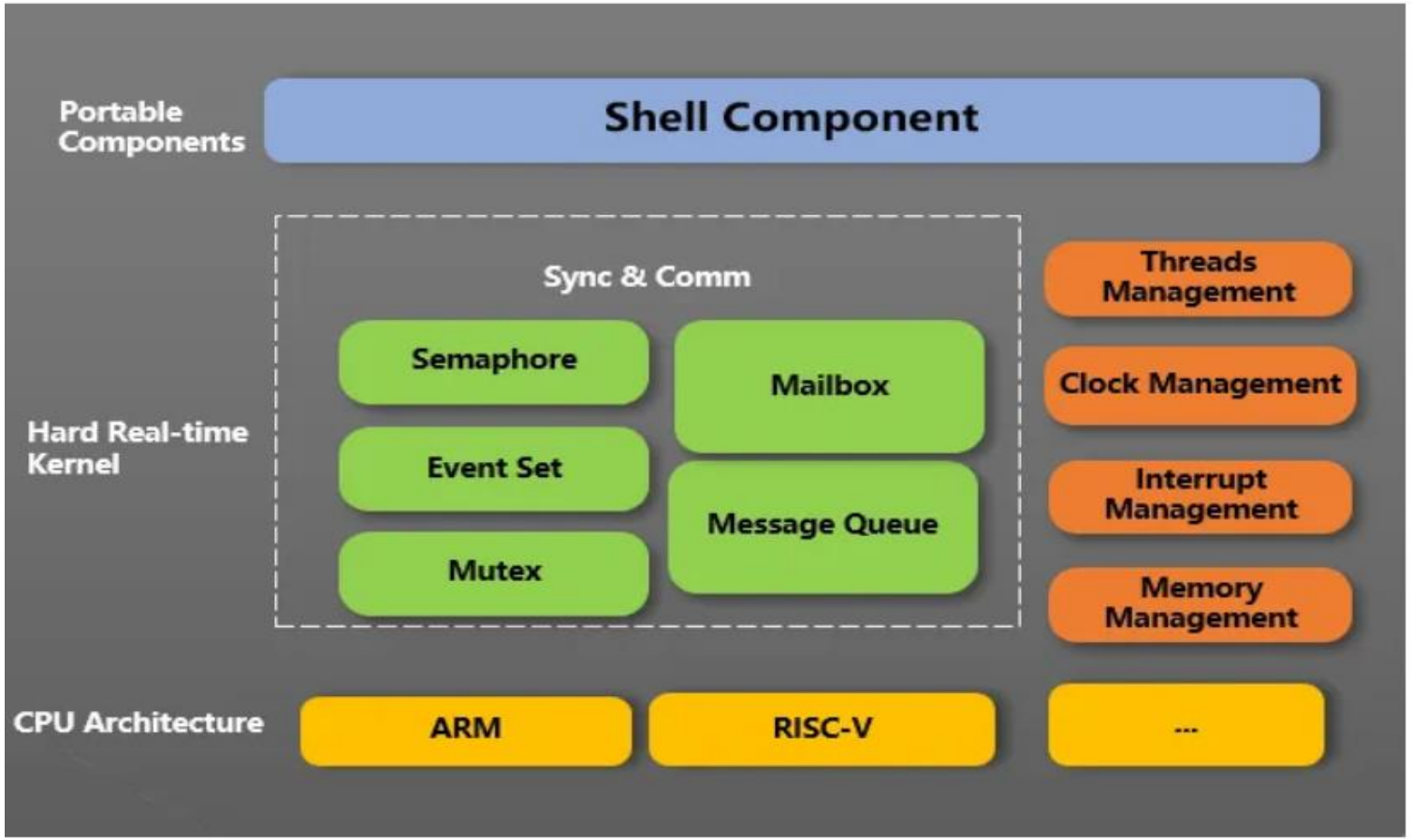
Real-time systems (low latency)

Security-critical applications (minimal attack surface)

Examples:

L4 Nanokernel, QNX, Zephyr OS

NanoKernels



Nano Architecture